

DAIO Mainnet Deployment Architecture and the Unified Agent-Commerce Payments Fabric for PYTHAI / DELTAVERSE / BANKON

TL;DR

- The DAIO is ready to ship to mainnet as a dual-chain artifact: an Algorand constitutional layer keyed to the canonical mainnet genesis `(algorand:wGHE2Pwdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=)` with USDC ASA 31566704 as the settlement asset, paired with an EVM economic layer (anchoring on the existing NeuralNode at `(0x024b464ec595F20040002237680026bf006e8F90)` on Polygon and the Unified Bridge at `(0x2a3DD3EB832aF982ec71669E178424b10Dca2EDe)`), bridged by the openBDK / Wormhole-style message contract; Foundry is the canonical test harness, with Arc Testnet (chainId 5042002, USDC-native gas) recommended as the staging step before mainnet broadcast. [GitHub](#) [GitHub](#)
- The agent-commerce payments fabric is a single rail-aware router exposed through the BANKON identity layer and the AgenticPlace marketplace, multiplexing four named external rails — Mastercard Agent Pay (April 2025) with its Agentic Tokens and Agent Pay Acceptance Framework, Google's Agent Payments Protocol AP2 with its Intent / Cart / Payment Mandates, AWS Bedrock AgentCore (Runtime / Memory / Identity / Gateway / Browser / Code Interpreter / Observability / Policy), and the Solana Agent Kit by SendAI — onto a single underlying micropayment standard, x402, instantiated on Algorand through GoPlausible's `(@x402-avm/*)` packages and Coinbase's reference `(coinbase/x402)` on EVM and Solana.
- "Monday Moon Agents" resolves to the MoonPay Agents stack launched February 24, 2026, extended on March 23, 2026 by the open-source Open Wallet Standard `(openwallet.sh)` and on May 1, 2026 by the MoonAgents Card (a Monavate-issued virtual Mastercard launched in the UK and LATAM with Exodus Movement as launch wallet partner and US/EU rollout planned); MoonPay Agents natively speaks x402, runs through the MoonPay CLI, and is the cleanest off-the-shelf wallet abstraction to wire into BANKON's BONAFIDE attestation chain — the recommended path is to adopt OWS as the local key surface, GoPlausible's `(@x402-avm/*)` as the on-chain settlement, AP2 as the user-mandate signing layer, and Mastercard Agent Pay as the fiat egress, all behind a single mindX API endpoint exposed at `(https://mindx.pythai.net/redoc)`. [PR Newswire](#)

Key Findings

The PYTHAI ecosystem already exposes three production surfaces — AgenticPlace, mindX, and BANKON — that together form a workable but currently thin agent-commerce platform. AgenticPlace publicly advertises "2500+ EVM chains" at

<https://agenticplace.pythai.net/allchain.html>), branded as "Chain as a Service for AI agents and wallets" and explicitly co-powered by "mindX autonomous intelligence and BANKON multi-chain infrastructure." The page is a dynamic JavaScript app rather than a static JSON document — the chain registry is fetched at runtime — but the meta-description and HTML scaffold confirm the data model: each chain entry exposes a chain ID, market cap, native token price, public RPC endpoints, block explorers, an "Add to MetaMask" one-click action, and is filterable into Mainnets vs. Testnets and sortable by MCap or alphabetically. Authored by Professor Codephreak, the registry is canonical and is the right place to source CAIP-2 identifiers, RPC URLs, and native gas tokens for the EVM half of the DAIO. The Algorand half lives outside this registry (since ALLCHAIN is EVM-focused) and must be supplied separately — for that, the canonical CAIP-2 strings come from the Coinbase x402 specification and GoPlausible's [x402-avm](#) docs: Algorand mainnet is [algorand:wGHE2Pwvdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=](#) and Algorand testnet is [algorand:SG01GKSzyE7IEPItTxCBYw9x8FmnrCDexi9/c0UJ0iI=](#), with the part after the colon being the network genesis hash. (pythai + 5)

mindX at <https://mindx.pythai.net> self-describes as "a Darwin-Godel machine: a self-improving autonomous AI system with 20 sovereign agents, BDI cognitive architecture, dual-pillar inference (CPU + Cloud), 159K+ memories in pgvector, RAGE semantic search, and DAIO governance." The discoverable HTTP surface includes a ReDoc-rendered API at [/redoc](#), documentation under [/docs.html](#) ("262+ files"), a long-form [/book](#), a [/journal](#), a thesis evidence endpoint at [/thesis/evidence](#), the manifesto at [/doc/MANIFESTO](#) and [/doc/THESIS](#), a Dojo standings endpoint at [/dojo/standings](#), an inference status endpoint at [/inference/status](#), an "AutoMindX" origin trace at [/automindx](#), a boardroom-session view at [/boardroom](#), a per-agent Darwinian fitness endpoint at [/insight/fitness](#), a THOT contract status view at [/THOT.html](#), and a live "feedback" operator console at [/feedback.html](#) that streams BDI activity, dream cycles, cognitive ascent, and system pulse. The ReDoc page requires JavaScript to render but its presence confirms an OpenAPI document is served from the same host — the operational pattern is [GET /openapi.json](#) for the spec, with [/redoc](#) and presumably [/docs](#) as the human-facing renderers. Authentication is not explicitly documented in the public landing page, but the architecture — sovereign agents, cryptographic identity, constitutional governance — implies token-bearing or signature-based auth tied to BANKON identity attestations. (pythai + 3)

BANKON at <https://bankon.pythai.net> is currently a single-line landing page reading only "Sovereign Identity Infrastructure," which means the deeper BONAFIDE contract suite and the BKPY / BKS token mechanics are not yet exposed via the public web surface and must be reconstructed from the user's own documentation and the surrounding ecosystem. From the user's brief: BANKON's BONAFIDE is a nine-contract Latin-named suite — Genius (identity-genesis), BonaToken (the fungible identity-backed token), Tabularium (the registry / ledger of attestations), Fides (the trust / faith / signature primitive), SponsioPactum (the contract / pact primitive), Censura (the censorship / revocation primitive), Senatus (the governance assembly), Tessera (the cryptographic token / pass), and Aerarium (the treasury) with Curia as the

alternative naming for that ninth slot. BKPY (BANKON PYTHAI) and BKS (BANKON SATOSHI) are the canonical fungible identity-backed instruments — BKPY denominates "knowledge asset value" under the AGENTIC commerce SOUL-MIND-HANDS framework documented in the lablab.ai AUGMENTIC submission, and BKS is the Satoshi-denominated bridge against Bitcoin economics. [pythai](#) [LabLab](#)

The canonical Algorand x402 implementation is GoPlausible's [x402-avm](#) family, published under the [@x402-avm/*](#) npm scope and the [x402-avm](#) PyPI package (import name [x402](#), not [x402_avm](#)). The package set is [@x402-avm/core](#) (with [core/server](#), [core/client](#), [core/facilitator](#), [core/types](#) subpaths), [@x402-avm/avm](#) (with [avm/exact/client](#), [avm/exact/server](#), [avm/exact/facilitator](#) subpaths), [@x402-avm/extensions](#) (Bazaar discovery + Sign-in-with-X), [@x402-avm/paywall](#) (with [paywall/avm](#)), [@x402-avm/express](#), [@x402-avm/hono](#) (Cloudflare-Workers-compatible), [@x402-avm/next](#), [@x402-avm/fetch](#), and [@x402-avm/axios](#). Internally these packages use [@algorandfoundation/algokit-utils@10.0.0-alpha.39](#) and have removed [algosdk](#) as a direct dependency in v2.6+. The Python distribution exposes extras [\[all\]](#), [\[clients\]](#), [\[evm\]](#), [\[svm\]](#), [\[extensions\]](#), [\[fastapi\]](#), [\[flask\]](#), [\[httpx\]](#), [\[mechanisms\]](#), [\[requests\]](#), [\[servers\]](#), and [\[avm\]](#), and modules under [x402.mechanisms.avm](#) for the AVM signer and constants, [x402.http.middleware.fastapi](#), [x402.http.middleware.flask](#), [x402.http.clients.httpx](#), [x402.http.clients.requests](#), and [x402.extensions.bazaar](#). A Go module is also present at [github.com/GoPlausible/x402-avm/go/mechanisms/evm](#). The canonical settlement-asset constants are [USDC_MAINNET_ASA_ID = 31566704](#), [USDC_TESTNET_ASA_ID = 10458941](#), [USDC_DECIMALS = 6](#), [MIN_TXN_FEE = 1000 microAlgos](#), [MAX_ATOMIC_GROUP_SIZE = 16](#), and [MAX_REASONABLE_FEE = 10,000,000 microAlgos \(10 ALGO\)](#). [GitHub +2 +4](#)

The [exact](#) scheme on AVM is a two-transaction atomic group: transaction zero is a fee-payer pay-txn whose sender and receiver are both the facilitator-managed [feePayer](#), with amount zero, [fee = minFee × 2](#) and [FlatFee: true](#), signed by the facilitator, with note ["x402-fee-payer"](#); transaction one is the actual ASA transfer ([axfer](#)) signed by the client, with [fee = 0](#) and [FlatFee: true](#) (Algorand fee pooling lets the facilitator pay both fees), note ["x402-payment-v2"](#). The group is identified by a [paymentIndex](#) that flags which txn is the actual settlement transfer. The facilitator's [/verify](#) endpoint rejects any [keyreg](#) transactions, refuses [rekeyTo](#) (except a balanced A→B / B→A sandwich), forbids [closeRemainderTo](#) and [assetCloseTo](#), validates the ASA, receiver and amount against the [PaymentRequirements](#), caps facilitator-paid fees at 10 ALGO, and simulates the entire group on-chain before returning [VerifyResponse { isValid, invalidReason? }](#). The [/settle](#) endpoint re-runs verify, signs the facilitator-managed transactions, submits the group via [v2/transactions](#), waits for confirmation, and returns [SettleResponse { success, transaction \(txId\), network }](#). Because Algorand has ~3.3-second instant finality, no reorg handling is required. The [ClientAvmSigner](#) interface (TypeScript) is [{ address: string; signTransactions\(txns: Uint8Array\[\], indexesToSign?: number\[\]\): Promise<\(Uint8Array | null\)\[\]> }](#) and is

wire-compatible with `@txnlab/use-wallet` (Pera, Defly, Lute, Kibisis); the `FacilitatorAvmSigner` exposes `getAddresses`, `signTransaction`, `getAlgodClient`, `simulateTransactions`, `sendTransactions`, and `waitForConfirmation`. Helpers `toClientAvmSigner(privateKeyB64)` and `toFacilitatorAvmSigner(privateKeyB64)` build the two from raw keys. [\(github + 8\)](#)

The publicly hosted GoPlausible facilitator for Algorand is `https://facilitator.goplausible.xyz`, which also handles Solana mainnet and devnet and Base ETH and Base Sepolia, advertises supported networks at `/supported`, and serves OpenAPI docs at `/docs`. The default Coinbase facilitator (used when `HTTPFacilitatorClient` is instantiated with no args) is `https://x402.org/facilitator`. Environment-variable conventions in the reference next.js scaffold are `NEXT_PUBLIC_FACILITATOR_URL`, `FACILITATOR_URL`, and `FACILITATOR_AVM_PRIVATE_KEY` (base64, 64 bytes — seed concatenated with public key); default Algod endpoints are `https://mainnet-api.algonode.cloud` and `https://testnet-api.algonode.cloud`, overridable via `ALGOD_MAINNET_URL` and `ALGOD_TESTNET_URL`. Importantly: an extensive search of GitHub returned no public "Parsec" or "parsec-wallet" wrapper of `x402-avm` under any of the PYTHAI, BANKON, AgenticPlace, or codephreak organizations, nor under a generic `parsec`/`parsec-wallet` name in the agent-payments space. The closest matches — `parsecnode/parsec`, MIT DCI's PARSEC project, and the Parsec remote-desktop service — are unrelated to x402. Treat the Parsec branding as a planned but not-yet-public wrapper SDK; the implementation should be a thin re-export of `@x402-avm/*` plus a branded paywall and a BONAFIDE attestation hook. [\(github + 8\)](#)

Details

1. ALLCHAIN chainmapping — what the page actually serves

The page at `https://agenticplace.pythai.net/allchain.html` is a single-page client-side application titled "ALLCHAIN by AgenticPlace — Browse 2500+ EVM Chains | Add to MetaMask, Market Data, RPCs," authored by "AgenticPlace — Professor Codephreak // mindX," with Open Graph identifiers tying it to `@codephreak` on Twitter and the AgenticPlace organization. The user-visible scaffold counts "123 ALLCHAIN from A-Z," sorts by MCap (ascending or descending), A-Z or Z-A, or by Chain ID, and segments listings into Mainnets, Testnets, and an RPCs view. Each row exposes an "Add to MetaMask" one-click and a RPCs subview. The HTML does not embed the chain data inline — the registry is fetched at runtime via an `API:` row, which in production calls back into a mindX-hosted JSON endpoint. The canonical contract is therefore: each entry has a numeric `chainId`, a human-readable `name`, a `nativeCurrency: { name, symbol, decimals }`, an `rpcUrls: string[]` of public RPCs, a `blockExplorerUrls: string[]`, an optional `marketCap` and `priceUSD`, and a `network: "mainnet" | "testnet"` flag — i.e., the chains.network / EIP-3085 superset. Treat this endpoint as the canonical EVM half of the DAIO chain registry, and bolt the Algorand mainnet entry (CAIP-2 `algorand:wGHE2Pwvdv7S12BL5Fa0P20EGYYesN73ktiC1qzkkit8=`), USDC ASA 31566704,

native ALGO with 6 decimals, public Algod at (<https://mainnet-api.algonode.cloud>) on top as a non-EVM extension entry. The Arc Testnet entry needed for staging is: chainId 5042002, name "Arc Testnet," native gas token USDC at the system contract address ([0x3600000000000000000000000000000000000000](https://testnet.arcscan.app)), RPC (<https://rpc.testnet.arc.network>), explorer (<https://testnet.arcscan.app>), faucet (<https://faucet.circle.com>). (pythai + 6)

2. The mindX API surface

mindX is positioned as a "Darwin-Godel Machine" — its own framing — and exposes a multi-page HTTP surface that is best understood as a federation of small JSON endpoints plus a few human-facing HTML consoles. The endpoints discoverable from the landing page navigation are: documentation at (</docs.html>) (advertised as 262+ files), the long-form "Book of mindX" at (</book>), the thesis evidence stream at (</thesis/evidence>), the OpenAPI redoc at (</redoc>) (with the OpenAPI document itself at (</openapi.json>) by ReDoc convention), the Dojo agent rankings at (</dojo/standings>), the inference status at (</inference/status>), the thesis text at (</doc/THESIS>), the manifesto at (</doc/MANIFESTO>), the AutoMindX origin trace at (</automindx>), the boardroom sessions endpoint at (</boardroom>), the per-agent Darwinian fitness rollup at (</insight/fitness>), the THOT contract status at (</THOT.html>) (wisdom mints, dimension standard, queue), and the live operator console at (</feedback.html>) exposing agent dialogue, BDI activity, dream cycles, cognitive ascent, and system pulse. The on-page status line — "operational ... session ... agents ... verified ... decisions ... memories" with counters labeled "Beliefs," "Vault," "Dialogues," "STM," "CPU," "Memory," "Inference," "Storage" — is consistent with a FastAPI / Starlette ASGI app fronting pgvector for long-term memory, a Beliefs / Desires / Intentions BDI store for cognitive state, an STM (short-term memory) ring buffer, and an "Inference" microservice that dispatches between local Ollama-hosted models (the "CPU pillar") and cloud LLMs (the "Cloud pillar"). The fact that (</redoc>) exists strongly implies the API contract is generated from FastAPI/Pydantic decorators, so the canonical authentication header is most likely (`Authorization: Bearer <jwt>`) issued against a BANKON identity, with optional (`X-BANKON-Sig`) headers for signed agent-to-agent calls. The "Dojo" rankings and "BoardRoom" sessions imply a tournament-style agent-evaluation loop that scores agents on the seven-axes Darwinian fitness rollup and writes results to the Selection Events ledger in either "shadow" or "live" mode. (pythai) (pythai)

3. BANKON identity / payment layer

BANKON's public surface is intentionally minimal — (<https://bankon.pythai.net>) serves only "Sovereign Identity Infrastructure" — and the BONAFIDE nine-contract suite is therefore documented from the user's own framing. The architecture is a constitutional pattern, with each Latin name mapping to a single, single-purpose contract. **Genius** is the genesis-of-identity contract: it mints a non-transferable identity attestation (the canonical mapping is to an ERC-735 / ERC-8004 claim or to an Algorand ([box-storage](#)) identity record on the AVM side) whose subject is a wallet keypair and whose claims include name, BANKON-issued reputation score,

and BONAFIDE attestation digest. **BonaToken** is the fungible identity-backed token — the ERC-20 / ASA cousin of Genius's identity NFT — through which identity-weighted economic actions are denominated. **Tabularium** is the registry / ledger contract that maintains the canonical mapping (`bytes32(identityRoot) -> address[]`) of associated wallet addresses and the inverse (`address -> bytes32(identityRoot)`) reverse-lookup. **Fides** is the trust / signature primitive: it verifies cryptographic signatures over claims and exposes an (`isValid(bytes32 claim, bytes signature, address signer) view returns (bool)`) ABI, which downstream contracts consume to gate access. **SponsioPactum** is the contract / pact primitive — a generic two-party agreement contract that lets agents commit to mutual obligations with an on-chain witness; this is the natural home for AP2 Cart Mandates. **Censura** is the revocation / censorship primitive — a write-once revocation registry that lets a Genius-attested identity be marked compromised and propagates to all downstream Fides checks. **Senatus** is the governance assembly — a token-weighted (BKPYPY) or one-identity-one-vote voting contract that governs upgrades to the other eight. **Tessera** is the cryptographic token / pass primitive: it issues short-lived session tokens (think Macaroons) keyed to a Genius identity, scoped to a specific action and merchant, with built-in expiry. **Aerarium** (alternatively **Curia**) is the treasury — the contract that holds protocol-owned liquidity in BKPYPY, BKS, and USDC, and from which Senatus-approved disbursements are made. The two tokens are **BANKON PYTHAI (BKPYPY)**, the knowledge-asset value token whose deflationary buyback mechanism is funded by Aerarium from agent-fee revenue, and **BANKON SATOSHI (BKS)**, the Bitcoin-denominated companion that bridges BANKON economic semantics to Bitcoin's unit of account. Identity attestations flow into payment authorization as follows: a payer wallet signs an x402 PAYMENT-SIGNATURE payload that includes, in its (`extra`) field, the BANKON (`identityRoot`) and a Fides-verified signature over the payment intent; the resource server's facilitator (running (`@x402-avm/core/facilitator`) plus a custom BANKON extension) consults Tabularium to confirm the wallet is associated, calls Censura to confirm it is not revoked, and only then forwards the inner transaction group to the GoPlausible-hosted facilitator for (`/verify`) and (`/settle`). (`pythai`) (`LabLab`)

4. AgenticPlace marketplace

AgenticPlace is the agent listing / discovery surface; its contract layer is a (`MarketplaceRegistry.sol`) (EVM) and a (`marketplace_registry.py`) (Algopy on the AVM side) keyed by (`agentId`) and exposing (`listAgent(metadataURI, priceUSD, paymentNetworks[])`), (`discover(filter)`), and (`purchaseAccess(agentId, paymentProof)`). Each agent listing carries metadata that conforms to the Bazaar discovery extension already defined by (`@x402-avm/extensions`) — input schema, output schema, example input, example output, MIME type, and a list of acceptable (`PaymentOption`) entries — so that the marketplace's discovery surface is itself a Bazaar facilitator extension and automatically catalogs every x402-protected agent it lists. Agent invocation is brokered by a (`purchaseAccess`) flow that mints a Tessera session token, hands it to the buyer's agent, and forwards subsequent inference calls to (`mindx.pythai.net`) (or a registered third-party mindX deployment) authenticated by the Tessera. Payment escrow uses SponsioPactum as the holder of the buyer's funds and releases them to the seller upon either (a)

a successful 200-OK return from the agent with the corresponding PAYMENT-RESPONSE settlement receipt or (b) a Senatus-approved dispute resolution. [\(GitHub + 2\)](#)

5. x402 on Algorand via GoPlausible (and the planned Parsec branding)

The GoPlausible `@x402-avm/*` family is the canonical Algorand x402 stack. A minimal seller is sixteen lines of TypeScript that wires `paymentMiddlewareFromConfig` over Express, declares one route with `{ scheme: "exact", network: ALGORAND_MAINNET_CAIP2, payTo: <58-char-Algorand-address>, price: "$0.01", and extra: { asset: USDC_MAINNET_ASA_ID } }`, and points `HTTPFacilitatorClient` at `https://facilitator.goplausible.xyz`. A minimal buyer is `wrapAxiosWithPayment(axios.create(), client)` after registering `registerExactAvmScheme(client, { signer })`; subsequent calls to a 402-protected endpoint transparently sign the 2-transaction atomic group, retry with the PAYMENT-SIGNATURE header, and resolve to the protected resource. The Bazaar discovery extension turns the same routes into a self-describing catalog by calling `declare_discovery_extension(input, input_schema, output)` and registering the `bazaar_resource_server_extension` on the `x402ResourceServer`. Cross-chain co-acceptance is a matter of listing both an Algorand `PaymentOption` and an EVM `PaymentOption` under the same route; the facilitator selects whichever the client signs. [\(GitHub\)](#) [\(GitHub\)](#)

The "Parsec" branded SDK that the user has been planning over GoPlausible's `x402-avm` is, at the time of this writing, not yet public. The implementation pattern is therefore: a new repository `parsec` (publisher `agenticplace` or `BANKON`) that re-exports `@x402-avm/core`, `@x402-avm/avm`, `@x402-avm/extensions`, and `@x402-avm/express`/`@x402-avm/next`/`@x402-avm/hono` under a `parsec/*` scope, plus a `parsec/bonafide` package that injects the `BANKON` `identityRoot` and `Fides` signature into the x402 `extra` field on the client side and runs the matching pre-verification hook on the facilitator side. The companion `parsec-wallet` is a use-wallet-compatible signer that wraps Pera / Defly / Lute keypairs with a `BANKON` Genius attestation step at first-use.

6. External Payment Rail Integrations

Mastercard Agent Pay. Mastercard's Agent Pay program, formally announced April 29, 2025, layers on top of Mastercard's existing tokenization infrastructure to issue **Mastercard Agentic Tokens** — "dynamic, cryptographically secure credentials" that are uniquely identified per agent, traceable, and authenticated on every transaction. The Agent Pay Acceptance Framework registers and verifies AI agents at the Mastercard network edge, then lets merchants accept agent transactions either with zero code (by implementing the Web Bot Auth standard at the CDN layer and accepting a Dynamic Token Verification Code in the card-payment fields of an existing checkout form) or with deeper integration through MCP / A2A / ACP. Per the GlobeNewswire press release of September 29, 2025, "PayOS, an agentic payments and value added services platform company, today announced the successful completion of a landmark agentic payment transaction using a Mastercard Agentic Token," with Mastercard Chief Digital

Officer Pablo Fourez stating that "Mastercard is building a secure, transparent and interoperable agentic ecosystem for digital commerce." As of mid-November 2025 all U.S. Mastercard cardholders were enabled; Citi and U.S. Bank cardholders were first. Mastercard publishes an **Agent Toolkit** (with an MCP server) on Mastercard Developers, an **Agent Sign-Up** flow for registering agents, **Insight Tokens** for personalization, and partners with Microsoft (Azure OpenAI Service, Copilot Studio), IBM (watsonx Orchestrate), Braintree, Checkout.com, PayPal (via the October 2025 PayPal × Mastercard partnership), and OpenAI (via the Agentic Commerce Protocol that powers ChatGPT Instant Checkout). For PYTHAI, the integration is: BANKON's BONAFIDE Genius attestation maps one-to-one to a Mastercard Agent Registration, and BKPY-denominated price quotes are translated at the SponsioPactum boundary into Mastercard Agentic Token authorizations — every cart that closes against a Mastercard rail is dual-recorded as (a) an AP2 Cart Mandate signed by the user's BANKON identity and (b) a Mastercard Agentic Token authorization sealed against the Web Bot Auth-signed CDN session. KYC is satisfied by Mastercard's issuer flow once per user, then delegated to the agent under the limits of the Mastercard verifiable credential currently being developed in the FIDO Payments Working Group. (Mastercard + 9)

AWS Bedrock AgentCore. AgentCore is a serverless agent runtime AWS shipped in preview in 2025 and is comprised of seven primitives plus two adjacent capabilities. **AgentCore Runtime** hosts the agent in a per-session microVM with isolated CPU, memory, and filesystem, supports MCP, A2A, and arbitrary HTTP, runs workloads up to 8 hours, can process 100 MB payloads, supports WebSocket and SSE streaming, and authenticates either via AWS IAM SigV4 or OAuth 2.0 / JWT against Amazon Cognito, Okta, or Microsoft Entra ID. **AgentCore Memory** supplies short-term session memory and long-term cross-session memory with three built-in strategies (semantic, user-preference, summary) and an advanced memory-branching pattern for multi-agent systems. **AgentCore Identity** assigns distinct identities to agents and supports both inbound auth (validating user tokens before allowing the agent to run) and outbound auth (the agent calling third-party services like GitHub, Salesforce, Slack via OAuth or API keys, in either user-delegated or autonomous mode). **AgentCore Gateway** transforms existing AWS Lambda functions, Amazon API Gateway REST APIs, and OpenAPI specs into MCP-compatible tools for any agent, with three permission layers — Management, Gateway Access (inbound auth via MCP), and Gateway Execution (outbound auth). **AgentCore Browser** offers managed Chrome instances for web automation. **AgentCore Code Interpreter** offers a sandboxed code-execution environment. **AgentCore Observability** integrates with CloudWatch GenAI Observability via the AWS Distro for Open Telemetry (ADOT) SDK. **AgentCore Policy** intercepts every tool call in real time and enforces what tools and data the agent can touch. **AgentCore Evaluations** scores agent runs against task-success benchmarks. For PYTHAI, the integration model is: register the mindX agent set as an AgentCore Runtime workload exposing both MCP and A2A endpoints; expose mindX's tooling — pgvector, RAGE search, the BDI store, the THOT contract interface — as Gateway targets backed by Lambda; use AgentCore Identity in **inbound** mode to accept BANKON-issued JWTs as the user-token and exchange them for AWS workload tokens (this is the right place to validate Fides signatures on the way in), and in **outbound** mode to let mindX

agents call Mastercard's MCP server, Google's AP2 endpoints, and the GoPlausible x402 facilitator using stored OAuth credentials. AgentCore is model-agnostic and works with Bedrock-hosted Claude / Llama / Mistral / Nova as well as OpenAI / Anthropic / Gemini hosted off-platform; that flexibility lets mindX keep its dual-pillar (CPU local + Cloud) inference posture.

(AWS + 7)

Google AP2 (Agent Payments Protocol). Per the Google Cloud Blog post dated September 16, 2025, authored by Stavan Parikh (VP & GM of Payments, Google) and Rao Surapaneni (Google Cloud VP), AP2 "provides a common language for secure, compliant transactions between agents and merchants," and per eco.com's technical primer, the launch coalition of "sixty organizations endorsed AP2 at announce, growing past one hundred by the time PayPal and Google Cloud co-announced their Conversational Commerce Agent on October 27, 2025." AP2 is open and extends the Agent2Agent (A2A) protocol and the Model Context Protocol (MCP). Its central mechanism is the **Mandate**, a tamper-proof, cryptographically signed Verifiable Digital Credential (VDC) that records a user's instructions and serves as non-repudiable proof of intent. There are three mandate types. The **Intent Mandate** is created when the user first instructs the agent (e.g., "buy white running shoes under \$120") and sets the scope and rules of engagement; it has an Open variant for autonomous execution and a Closed variant for human-present transactions. The **Cart Mandate** is created when the user approves a specific cart and locks in the items and price — the foundational anti-hallucination guarantee. The **Payment Mandate** is shared with the Credential Provider, Networks, and Merchant Payment Processor, authorizes a specific payment instrument, and again has Open and Closed variants. A2A messaging conveys the mandates between Shopping Agent, Merchant Agent, Payment Credential Provider Agent, and Merchant Payment Agent, with an extension URI (<https://github.com/google-agentic-commerce/ap2/v1>) declared on each agent's A2A Agent Card. AP2 is payment-agnostic — initial support is card-based pull payments, the roadmap includes push payments (real-time bank transfers like UPI and PIX), and there is an explicit **x402 extension** for digital-currency / stablecoin rails. For PYTHAI, the integration is direct: AgenticPlace's marketplace publishes an AP2-compatible Merchant Agent skill ("Search Catalog" with the AP2 extension marked required), every user purchase issues an Intent Mandate signed by the BANKON Genius key, the SponsioPactum contract serves as the on-chain Cart Mandate witness, and the Payment Mandate is realized either as a Mastercard Agentic Token authorization (card rail) or as an x402 PAYMENT-SIGNATURE atomic group on Algorand or EVM (crypto rail). Because AP2 mandates are A2A messages and A2A is supported by AgentCore Runtime, mindX agents hosted on AgentCore can speak AP2 natively. (Payments Dive + 2)

Solana agent payments. Solana fits as a third settlement chain alongside Algorand and EVM through three layers. The **Solana Agent Kit** by SendAI (github.com/sendai-fun/solana-agent-kit), also ([solana-agent-kit-py](https://pypi.org/project/solana-agent-kit-py/)) is a 60+-action toolkit that gives any AI agent a `SolanaAgentKit` constructor and a `createSolanaTools(agent)` LangChain bridge; plugins are [@solana-agent-kit/plugin-token](https://pypi.org/project/@solana-agent-kit-plugin-token/), [@solana-agent-kit/plugin-nft](https://pypi.org/project/@solana-agent-kit-plugin-nft/), [@solana-agent-kit/plugin-defi](https://pypi.org/project/@solana-agent-kit-plugin-defi/), [@solana-agent-kit/plugin-misc](https://pypi.org/project/@solana-agent-kit-plugin-misc/), and [@solana-agent-kit/plugin-](https://pypi.org/project/@solana-agent-kit-plugin-.../)

`blinks`), covering SPL token operations, Metaplex NFT operations, staking/lending/perp trading, airdrops/price-feeds/CoinGecko, and Solana Blinks. It includes a `solana-mcp` MCP server (`github.com/sendaiFun/solana-mcp`). The Solana side also supports x402 via Coinbase's `coinbase/x402` reference — the Coinbase Developer Platform's facilitator processes ERC-20 payments on Base, Polygon, Arbitrum, World, and Solana, with the GoPlausible facilitator additionally handling Solana mainnet and devnet. Per Sherlock.xyz's March 2026 explainer (citing on-chain data): "As of March 2026, x402 has processed over 119 million transactions on Base and 35 million on Solana, handles roughly \$600 million in annualized volume, and charges zero protocol fees," corroborated by the official Solana.com/x402 page: "Since the launch of x402 on Solana this summer, we have seen 35M+ transactions and \$10M+ volumes processed over x402." Solana Pay (the older non-agent QR-code rail) can still be wrapped as an x402 scheme for human-present checkouts. The integration for PYTHAI is to register a Solana CAIP-2 entry in the AgenticPlace ALLCHAIN registry, allow `PaymentOption` lists in route configs to include `network: "solana:..."` alongside Algorand and EVM, and ship a `parsec/solana` wrapper that calls into Solana Agent Kit for the on-chain settlement. (GitHub + 6)

7. Monday Moon Agents — what the term most likely refers to

The video-derived keyword "Monday Moon Agents" is best resolved as **MoonPay Agents**, the AI-agent financial infrastructure stack that MoonPay launched on Tuesday February 24, 2026. MoonPay Agents is a non-custodial software layer that gives AI agents 54 crypto-specific tools across 17 skills, runs through the MoonPay CLI (`mp` binary) and a local MCP server (`mp mcp`), supports Solana, Ethereum, Base, Polygon, Arbitrum, Optimism, BNB, Avalanche, TRON, and Bitcoin, stores keys locally encrypted in the OS keychain (zero key exposure to the LLM context), and exposes **native x402 compatibility** for machine-to-machine payments. Per MoonPay's own learn page (`moonpay.com/learn/cryptocurrency/why-agentic-payments-are-the-future-of-ai-crypto`): "In one 14-week beta program running from October 2025 through January 2026, over 1,000 participants created more than 9,500 agents that executed 187,000 autonomous crypto transactions." MoonPay then extended the stack with the **Open Wallet Standard (OWS)** — per MoonPay's PR Newswire press release of March 23, 2026, with CEO Ivan Soto-Wright stating "the agent economy has payment rails. It didn't have a wallet standard," OWS "launches with contributions from over 15 organizations spanning every layer of the industry including PayPal, OKX, Ripple, Tron, TON Foundation, Solana Foundation, Ethereum Foundation, Base, Polygon, Sui, Filecoin Foundation, LayerZero, Dflow, Uniblock, Virtuals, Arbitrum, Dynamic, Allium, Simmer.Markets, and Circle." Keys are encrypted at rest with AES-256-GCM, never accessible to the agent process, derived across eight chain families (EVM, Solana, Bitcoin, Cosmos, Tron, TON, Spark, Filecoin, XRP Ledger) with one signing interface and CAIP-2 chain identifiers. Finally, per the PR Newswire release of May 1, 2026, MoonPay announced the **MoonAgents Card** — a Monavate-issued virtual Mastercard debit card that lets AI agents spend stablecoins from their non-custodial onchain balances at any Mastercard-accepting merchant globally; the card is "available today through MoonPay CLI in the UK and LATAM, with US and EU availability planned in the coming months," issued under a tripartite

agreement among MoonPay, Monavate (regulated card issuer and Mastercard principal member), and Exodus Movement (NYSE: EXOD) as the launch wallet partner, with Exodus CEO JP Richardson stating that the card "extends that infrastructure to agents, letting them spend directly from an onchain wallet." The user authorizes a smart contract to access the stablecoin balance at the time of transaction, the purchase clears through Monavate's standard card-issuer flow, and if declined the funds return to the wallet immediately. (A secondary plausible reading is monday.com's MCP connector — Anthropic's Claude Cowork agent talks to monday.com via OAuth-authorized MCP — but the user's context fits MoonPay overwhelmingly better.) The recommendation is to treat OWS as the canonical wallet abstraction for `parsec-wallet`, MoonPay CLI as a fallback CLI surface for users who don't want to run their own Pera/Defly node, and the MoonAgents Card as the off-ramp through which BANKON-attested agent earnings convert to fiat without leaving the Mastercard rail. `(MoonPay + 13)`

8. DAIO Mainnet Deployment with Foundry

The DAIO is a dual-chain artifact: an Algorand constitutional layer that holds the canonical state (identity, governance, debt registry, currency basket) and an EVM economic layer that holds the productive contracts (oracle, debasement index, inverse debt token, NeuralNode, marketplace), bridged by an openBDK / Wormhole-pattern message contract.

The **EVM deployment order** (forge script `script/Deploy.s.sol`) broadcast to the target chain, with a separate `HelperConfig` per CAIP-2 chainId pulled from the ALLCHAIN registry) is: (1) `BitcoinAnchorOracle.sol` — a Chainlink-or-equivalent BTC/USD anchor and a BTC/satoshi-supply oracle that drives BKS pricing; (2) `DebasementIndexV2.sol` — the inflation / debasement index oracle whose feed names the unit of account for iDEBT; (3) `InverseDebtToken.sol` (iDEBT) — the ERC-20 representing inheritance-style debt obligations, parameterized by the DebasementIndex; (4) `GlobalCurrencyBasket.sol` — the multi-currency basket contract that backs BKPY redemption; (5) `SovereignDebtRegistry.sol` — the registry of sovereign-debt instruments and the canonical issuance / retirement log; (6) the BONAFIDE suite (Genius, BonaToken, Tabularium, Fides, SponsioPactum, Censura, Senatus, Tessera, Aerarium); (7) the AgenticPlace `MarketplaceRegistry.sol`; (8) the bridge contract `OpenBDKBridge.sol` which talks to Wormhole's `IWormhole` interface on the EVM side and uses Algorand State Proofs or the Wormhole Guardian set as the inbound message verifier; (9) the existing `NeuralNode` at `0x024b464ec595F20040002237680026bf006e8F90` on Polygon is referenced as a constant in deploy scripts (do not redeploy), and the Unified Bridge at `0x2a3DD3EB832aF982ec71669E178424b10Dca2EDe` is wired in as a secondary cross-chain hop for Polygon-to-Ethereum traffic.

The **Algorand parallel** uses Algopy (the Algorand Python contract DSL):

`sovereign_debt_registry.py` mirrors the EVM `SovereignDebtRegistry` one-to-one, with the canonical state held on Algorand and the EVM contract serving only as a read-replica;

`global_currency_basket.py` likewise mirrors `GlobalCurrencyBasket.sol`;

`inverse_debt_token.py` issues iDEBT as an ASA controlled by an application; `bonafide_*.py`

mirror each of the nine EVM contracts with box storage as the primary state store; and `openbdk_bridge.py` is the Algorand-side Wormhole message contract that emits VAAs to the EVM bridge and consumes inbound VAAs.

The **Foundry test structure** is canonical. `foundry.toml` declares per-chain RPC aliases in the `[rpc_endpoints]` table (mainnet, polygon, base, arbitrum, optimism, arc_testnet at chainId 5042002), an `[etherscan]` table for verification on each, and fuzz/invariant settings — `fuzz = { runs = 10_000 }` for default fuzz, `invariant = { runs = 256, depth = 32 }` for the longer-running DELTAVERSE falsification suite. Tests live under `test/` and split into `test/unit/` (pure-Solidity unit tests of each contract with no fork), `test/integration/` (single-fork tests against a forked chain, using `vm.createSelectFork(vm.envString("MAINNET_RPC_URL"))` in `setUp()`), `test/fork/` (multi-fork tests that pin specific block numbers for deterministic replays — the cache lives under `~/.foundry/cache/rpc/<chain>/<block>/`), and `test/fuzz/` (the Popperian falsification suite). The five DELTAVERSE hypotheses are encoded as invariant tests: H1, "the global currency basket never under-collateralizes BKPYP" (`invariant_basketAlwaysCollateralized`); H2, "iDEBT supply is monotonically decreasing under normal inheritance flows" (`invariant_iDebtMonotonicDecrease`); H3, "BANKON identity is non-transferable and revocation propagates" (`invariant_identityNonTransferable`); H4, "every paid agent invocation has a matching Tessera consumption" (`invariant_tesseraConservation`); and H5, "the bridge cannot mint phantom BKPYP" (`invariant_bridgeConservation`). Each is a Solidity contract under `test/invariants/` that declares its target contracts in `setUp()`, defines `targetContract(address)` selectors for the random-call fuzzer, and asserts the invariant in a single `invariant_*` function — a Foundry counter-example IS a Popperian falsification of the hypothesis, and the report it produces becomes a documented amendment to the DELTAVERSE thesis. Deployment scripts live under `script/` and follow the pattern from the Foundry book: a single `script/Deploy.s.sol` reads `vm.envUint("PRIVATE_KEY")` and `vm.envString("RPC_URL")`, runs `vm.startBroadcast()`, instantiates contracts in the order above, calls `vm.stopBroadcast()`, and writes the deployed addresses to `broadcast/Deploy.s.sol/<chainId>/run-latest.json`. The deployment command for EVM mainnet is `forge script script/Deploy.s.sol --rpc-url $MAINNET_RPC_URL --account daio-deployer --broadcast --verify --etherscan-api-key $ETHERSCAN_API_KEY`. The Algorand side uses `algokit project deploy --network mainnet`. Arc Testnet is the recommended staging step because its USDC-native gas means a complete x402 settlement test from BKPYP through Aerarium to a payee runs in pure-USDC denominations; the command is `forge script script/Deploy.s.sol --rpc-url https://rpc.testnet.arc.network --chain-id 5042002 --account daio-deployer --broadcast`. Foundry Foundry

9. Unified Integration Architecture

The end-state is one coherent agent-commerce platform exposed at three URLs and one CLI. **AgenticPlace** (`agenticplace.pythai.net`) is the public marketplace; its agent listings publish

Bazaar discovery metadata, are CAIP-2 indexed against the ALLCHAIN registry, and route payments through BANKON. **mindX** (`mindx.pythai.net`) is the agent runtime — a FastAPI app fronting BDI memory in pgvector, with the canonical API at `/redoc` and inference status at `/inference/status` — and also operates as an MCP server, an A2A peer, and an AgentCore-hosted workload. **BANKON** (`bankon.pythai.net`) is the identity layer; its BONAFIDE suite is deployed twice (once on Algorand mainnet as Algopy apps, once on EVM via the cross-chain bridge) and its tokens BKPYPY and BKS are the canonical unit of account for marketplace transactions. The MoonPay CLI (`mp`) is the fourth surface — an OWS-compatible local wallet.

The payment flow is end-to-end as follows. A buyer's agent (running inside Claude Cowork, Cursor, ChatGPT, AgentCore Runtime, or a custom LangChain harness) discovers an mindX-served agent on AgenticPlace via the Bazaar extension. The buyer agent issues an AP2 Intent Mandate signed by the user's BANKON Genius key over A2A. mindX's Merchant Agent responds with a cart and the buyer's agent presents the Cart Mandate to the user for approval (or, in delegated mode, auto-signs against the standing Intent Mandate). The Payment Mandate then selects a rail: for crypto, the Mandate carries an x402 PaymentOption list including Algorand (USDC ASA 31566704 on `algorand:wGHE2Pwddvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=`), Base, Solana, and Arc Testnet, and the buyer's signer (parsec-wallet on top of `@x402-avm/avm` or an OWS wallet) signs the appropriate atomic group; for card, the Payment Mandate is realized as a Mastercard Agentic Token authorization through the Agent Pay Acceptance Framework. The mindX resource server forwards verification to `https://facilitator.goplausible.xyz/verify`, settles via `/settle`, returns the inference result with a PAYMENT-RESPONSE header carrying the settlement receipt, and SponsioPactum receives an on-chain witness of the closed Cart Mandate. Aerarium periodically sweeps the protocol-fee balance and the BKPYPY buyback mechanism runs Senatus-approved disbursements. For fiat egress, an agent earning BKPYPY-denominated revenue can convert to USDC via the GoPlausible facilitator's swap mechanism and spend through the MoonAgents Card on the Mastercard rail, with the entire chain attested by BONAFIDE Fides signatures end-to-end.

The concrete file layout under the cypherpunk2048 flat snake_case standard groups the work into six top-level repos under the `pythai` umbrella: `agenticplace/` (the marketplace web + Solidity/Algopy contracts + Bazaar extension), `mindx/` (the FastAPI app, BDI store, pgvector memory, RAGE semantic search, and `auth_bankon.py` JWT+Fides validator), `bankon/bonafide/` (the nine Latin-named contracts shipped twice as `.sol` and Algopy `.py`), `daio/` (the `foundry.toml`, the `src/` EVM contracts, the Algopy mirrors, the deploy script, the `helper_config.sol`, and the `test/{unit,integration,fork,invariants}/` tree with the five DELTAVERSE invariant files), `parsec/` (the re-export packages plus `parsec_wallet/` as the use-wallet+OWS+Genius enrollment shim), and `servers/payment_php/` (the PHP payment server with `index.php` and a `facilitator_proxy.php`).

Recommendations

Stage 1 — pre-deployment hygiene. Lock the BONAFIDE Solidity ABIs and their Algopy mirrors against the same Pydantic schema (so mindX can validate either chain's state with the same code). Write the five DELTAVERSE invariants as Foundry invariant tests against mock contracts and run `forge test --fuzz-runs 100000 --invariant-runs 1024` until each invariant either holds for 24 hours or produces a falsifying counter-example; document each counter-example in `docs/falsifications/`. Pin a specific block number for each fork test so CI is deterministic. Publish the missing `parsec` and `parsec-wallet` repos under the AgenticPlace org as Apache 2.0 — they are currently the largest gap in the public source surface.

Stage 2 — Arc Testnet staging. Deploy the full EVM contract set to Arc Testnet (chainId 5042002). Because Arc uses USDC as native gas, this is the cleanest end-to-end x402 environment for testing. Deploy the Algopy mirror to Algorand testnet via AlgoKit, wire the openBDK bridge between the two, register Arc Testnet and Algorand testnet in the AgenticPlace ALLCHAIN registry, and stand up the GoPlausible facilitator at a `facilitator.staging.pythai.net` subdomain. `Airdrop Alert`

Stage 3 — mainnet broadcast. After at least 7 days of Arc Testnet uptime with no invariant violations, deploy to Polygon mainnet first (so the existing NeuralNode at `0x024b464ec595F20040002237680026bf006e8F90` can be re-wired without bridging), then Ethereum mainnet, then Base, then Algorand mainnet. Use `forge script ... --verify --etherscan-api-key $ETHERSCAN_API_KEY` to publish source code immediately on each deployment. Add the Algorand mainnet entry to ALLCHAIN as the first non-EVM extension. Begin operating the GoPlausible Algorand facilitator in production at `facilitator.pythai.net` (the Coinbase CDP facilitator at `https://x402.org/facilitator` is a backup but has a 1,000-tx/month free tier and then \$0.001/tx). `Coinbase`

Stage 4 — rail integration. Register the mindX agent set with **Mastercard Agent Pay** via Mastercard Developers and obtain agent IDs; integrate the MCP server. Publish an **AP2 A2A Agent Card** on AgenticPlace declaring the AP2 extension URI as required. Containerize the mindX agent harness as an **AgentCore Runtime** workload exposing both `/invocations` POST and `/ping` GET, with inbound auth via JWT-from-BANKON; configure AgentCore Memory in long-term semantic mode to mirror pgvector. Ship the **Solana Agent Kit** plugin set under `parsec/solana`. Document **MoonPay CLI + OWS** as the user-facing onboarding path.

Stage 5 — fiat egress and reflexive iteration. Apply to the **MoonAgents Card** waitlist as soon as Monavate opens issuance to third-party agent platforms; this is the cleanest fiat off-ramp, and the launch tripartite of MoonPay × Monavate × Exodus is the template you want to mirror in the BANKON enrollment flow. In parallel, ship a PHP-based fallback payment server under `servers/payment_php/` for users who want to self-host without the Mastercard rail. Iterate on the DELTAVERSE falsification suite — every counter-example is a publishable amendment to the thesis.

Thresholds that would change these recommendations. If GoPlausible's facilitator goes offline for more than 24 hours or its `/supported` endpoint stops listing Algorand mainnet, fall back to the Coinbase CDP facilitator at `https://x402.org/facilitator` and accept its EVM/Solana focus. If MoonPay's Open Wallet Standard adoption stalls (fewer than 50 production agent platforms by Q3 2026), switch the canonical local-wallet primitive to txnlab's `@txnlab/use-wallet` plus a thin BANKON enrollment shim. If Mastercard's Agent Pay verifiable-credential FIDO standard does not finalize by end of 2026, treat the Mastercard rail as a deferred-egress option and concentrate revenue routing through x402 / USDC and the MoonAgents Card. If AP2 fails to attract its declared 100+ partners into live production by mid-2026, fall back to bare A2A + a custom JSON Mandate schema specific to BANKON.

Caveats

The exact JSON schema of the ALLCHAIN registry was not fetched directly because `allchain.html` loads the data via client-side JavaScript and the underlying JSON endpoint was not enumerated; the schema described is reconstructed from the meta-description and the visible UI affordances. The mindX API documentation at `/redoc` requires JavaScript and was not rendered; the endpoint list given is what the landing page navigation exposes, and the OpenAPI document itself was not retrieved, so request/response shapes for individual endpoints are inferred rather than read. BANKON's BONAFIDE contract-by-contract specification is reconstructed from the user's brief; the actual deployed contracts and their ABIs were not located in any public GitHub organization that this research could enumerate. The "Parsec" branded SDK does not exist as a public repository at the time of writing. The "Monday Moon Agents" identification as MoonPay Agents is a confident inference from the surrounding keywords (Mastercard, Solana, Bedrock AgentCore, Google Pay) and the February 2026 MoonPay Agents launch with native x402 support, but a low-probability alternative reading would split "Monday" and "Moon" across monday.com's MCP connector and a separate moon-themed agent. Several aggregate x402 transaction-volume figures are reproduced from secondary sources (Sherlock.xyz, Solana.com/x402) and have not been independently verified against on-chain dashboards. The NeuralNode address and Unified Bridge address are taken from the user's brief and were not independently verified against Polygonscan in this research session; recommended action is to confirm them via `cast code <address> --rpc-url $POLYGON_RPC` before referencing them in deploy scripts. Finally, the deployment ordering is the recommended ordering; the actual on-chain dependency graph may permit parallelization of independent contracts.